



ORACLE
NetSuite

Financial Institution Parser Plug-in

2025.2

December 17, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Financial Institution Parser Plug-in Interface Overview	1
Financial Institution Parser Workflows	2
Developing a Financial Institution Parser Plug-in	3
Enabling Features for a Financial Institution Parser Plug-in	3
Creating a Financial Institution Parser Plug-in Script File	4
Administering a Financial Institution Parser Plug-in	6
Enabling Features for a Financial Institution Parser Plug-in	6
Adding the Plug-in Implementation	7
Conditions for Running a Financial Institution Parser Plug-in	8
Testing a Financial Institution Parser Plug-in	8
Error Propagation	9
Financial Institution Parser Plug-in Interface Definition	10
parseData()	10
inputData	11
pluginConfiguration	11
createAccountData()	12
getStandardTransactionCodes()	15
createNewStandardTransactionCode()	16
getExpenseCodes()	17
createNewExpenseCode()	17
getConfigurationPageUrl()	18
pluginConfiguration	18
configurationPageUrl	19

Financial Institution Parser Plug-in Interface Overview

Important: The Financial Institution Parser Plug-in interface uses SuiteScript 2.0. Your entry point scripts and supporting library scripts must also use SuiteScript 2.0. For information, see the help topics [SuiteScript 2.x Script Creation Process](#) and [SuiteScript 2.x](#).

Use a Financial Institution Parser Plug-in implementation to parse and retrieve the following:

- Bank and credit card data for matching and reconciliation
- Corporate card charges for expense reporting

Financial Institution Parser Plug-ins enable you to create and upload your own parsers for different bank and credit card statement formats and store them in NetSuite. The plug-in translates the information from data formats not supported by NetSuite's default parsers into financial data NetSuite can understand and use. This enables you to import statements in additional data formats, and then use the appropriate downstream feature, such as bank data matching and reconciliation or expense reporting.

To create a Financial Institution Parser Plug-in, you need to create the plug-in implementation script file, and then upload the script file and other utility files as required. For details, see [Creating a Financial Institution Parser Plug-in Script File](#).

You can define alternative implementations that suit your needs by customizing the core plug-in logic. To accommodate this process, NetSuite provides an interface definition that describes the name, parameters, and return type of each function defined in the plug-in's interface. For details, see [Financial Institution Parser Plug-in Interface Definition](#).

To use a Financial Institution Parser Plug-in, you need to create or edit a format profile record. The plug-in is compatible with both the Bank Reconciliation and Corporate Card Expenses format profile type.

Note: To import corporate card expense data, you must use a Financial Institution Parser Plug-in.

If you are configuring an automatic bank data import, you must use a Financial Institution Parser Plug-in with a Financial Institution Connectivity Plug-in. Together, these enable you to configure a direct connection to your data source so you can retrieve and parse financial data.

Note: You can use a Financial Institution Parser Plug-in to configure automatic or manual import of bank or credit card data for reconciliation. If you are configuring corporate card expense data imports for expense reporting, you should only use the automatic import method.

For details on configuring financial institution records and associated format profiles, see the help topic [Creating Financial Institution Records](#).

To run and test a Financial Institution Parser Plug-in, see [Conditions for Running a Financial Institution Parser Plug-in](#) and [Testing a Financial Institution Parser Plug-in](#).

A Financial Institution Parser Plug-in is supported by the SuiteCloud Development Framework (SDF). SDF enables you to customize your NetSuite account from an integrated development environment (IDE) on your local computer. For details, see the help topic [SuiteCloud Development Framework](#).

You can customize a Financial Institution Parser Plug-in in SDF by defining fiparserplugin objects. For details, see the help topic [Financial Institution Parser Plug-ins as XML Definitions](#).

For more information about Financial Institution Parser Plug-ins, see the following topics:

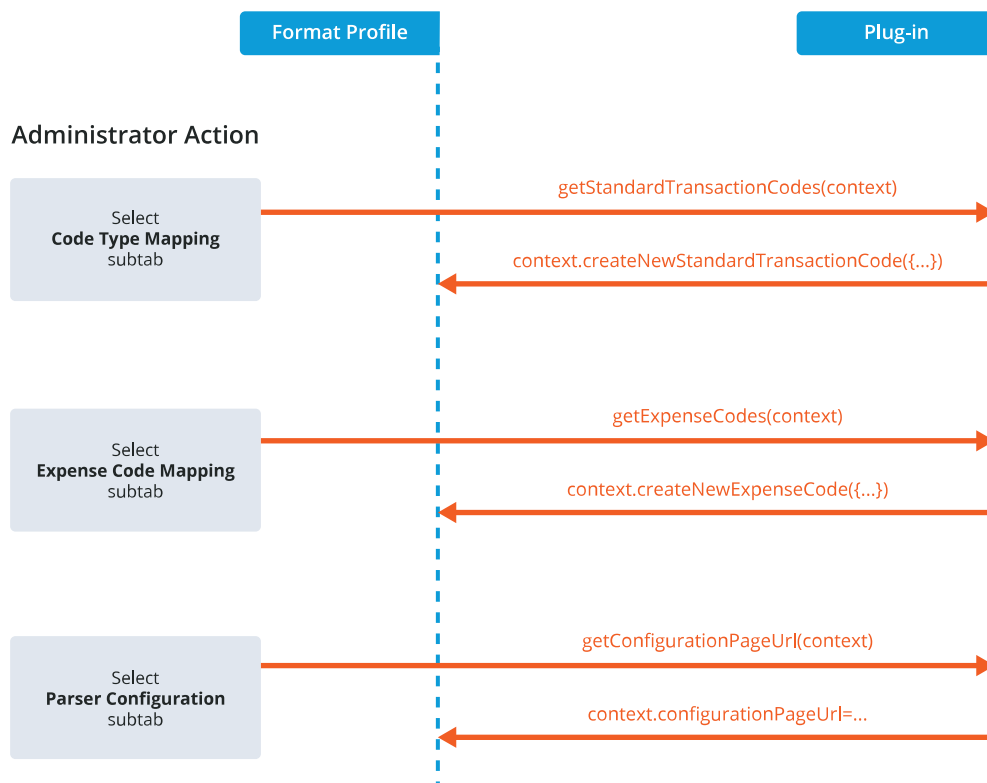
NetSuite Role	Help Topics
All roles	Financial Institution Parser Workflows
Developer	Developing a Financial Institution Parser Plug-in
Administrator	Administering a Financial Institution Parser Plug-in

In addition to using the API specifications to create a plug-in, you can use a default implementation of the plug-in called the AMEX Parser Plug-in. This plug-in is bundled with the AMEX Connectivity Plug-in and is provisioned by the AMEX Corporate Card Integration SuiteApp. Use these plug-ins if you want to import corporate card charges from American Express and don't want to write your own plug-ins. For details, see the help topic [American Express Integration for Expense Reporting](#).

Financial Institution Parser Workflows

A Financial Institution Parser Plug-in is invoked when you configure a format profile record and during the bank data import process.

The following diagram shows the workflow for the format profile record:



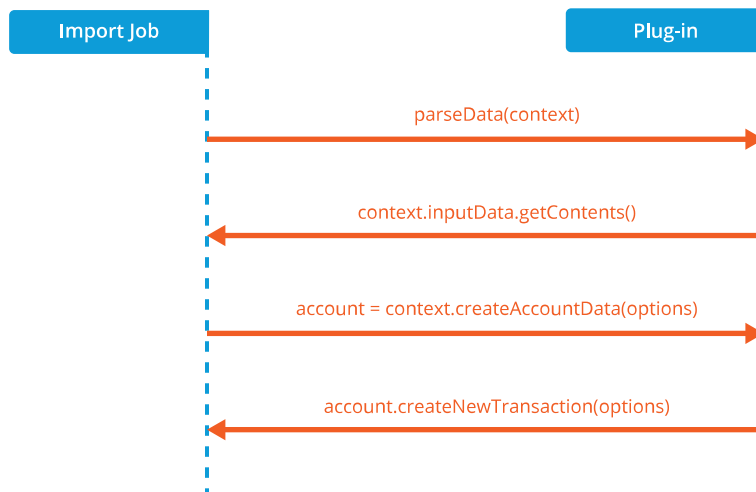
When you create a format profile and select a Financial Institution Connectivity Plug-in, or edit a format profile that uses that plug-in type, NetSuite calls an interface function. The format profile type dictates which interface function is called:

- If you're setting up a Bank Reconciliation format profile, NetSuite calls the `getStandardTransactionCodes()` interface function. This retrieves custom transaction codes from the plug-in, which are displayed on the Code Type Mapping subtab.

- If you are configuring a Corporate Card Expenses format profile record, NetSuite calls the `getExpenseCodes()` interface function to retrieve custom expense codes from the plug-in. These expense codes show up on the Expense Code Mapping subtab, which replaces the Code Type Mapping subtab.
- If the plug-in selected in a format profile record implements the `getConfigurationPageUrl()` interface function, NetSuite retrieves a URL for configuring the plug-in. This URL appears on the Parser Configuration subtab on the Format Profile page.

For more details, see the help topic [Format Profile Creation](#).

The following diagram shows the workflow for the bank data import process:



When you import data, NetSuite calls the `parseData()` interface function to get and transform a data file into account and transaction data. The plug-in does the following:

- Accesses the input data to be parsed, using the `inputData` interface object.
- Creates a new account data set, using the `createAccountData()` object function.
- Adds a new imported transaction to the account data set, using the `createNewTransaction()` object function.

Developing a Financial Institution Parser Plug-in

To develop a Financial Institution Parser Plug-in, complete the following steps:

- [Enabling Features for a Financial Institution Parser Plug-in](#)
- [Creating a Financial Institution Parser Plug-in Script File](#)
- [Conditions for Running a Financial Institution Parser Plug-in](#)
- [Testing a Financial Institution Parser Plug-in](#)
- [Error Propagation](#)

Enabling Features for a Financial Institution Parser Plug-in

To begin development or administration for a Financial Institution Parser Plug-in, make sure that the Server SuiteScript feature is enabled on the development account.

To enable features for the plug-in:

1. Go to Setup > Company > Enable Features.
2. On the **SuiteCloud** subtab, make sure **Server SuiteScript** is checked. If necessary, check the box and agree to the Terms of Service.
3. Click **Save**.

Creating a Financial Institution Parser Plug-in Script File

You must implement each Financial Institution Parser Plug-in interface function in a Javascript file (with a .js extension). You can use the SuiteCloud IDE, another Javascript IDE, or even a text editor to create your plug-in script.

This plug-in uses SuiteScript 2.0.

The following is a Financial Institution Parser Plug-in sample:

Note: You should define a custom record to store your configuration for setting up a bank connection. You should then deploy a Suitelet to create a custom implementation of a UI to save and load the record. The Suitelet lets you implement a custom interface that enables you to give NetSuite access to your bank accounts.

When returning a URL to the deployed Suitelet (using the `getConfigurationPageUrl` method), do not hardcode the URL. Instead, use the standard `resolveScript` SuiteScript method to define the structure of the URL, which includes specifying the script ID, deployment ID, and parameters.

For information about Suitelets, see the help topic [SuiteScript 2.x Suitelet Script Type](#). To build a UI in a Suitelet, see the help topic [N/ui/serverWidget Module](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType fiParserPlugin
4   */
5  define(['N/query', 'N/url'],
6    function (query, url)
7    {
8      // internal function used to load configuration for this plugin from a custom record
9      function loadConfiguration(configurationId)
10     {
11       var queryObj = query.create({
12         type: 'customrecord_sampleconfig',
13         columns: [{ fieldId: 'custrecord_memoprefix' }],
14         condition: {
15           fieldId: 'custrecord_configurationid',
16           operator: query.Operator.IS,
17           values: [configurationId]
18         }
19       });
20
21       return queryObj.run().results[0];
22     }
23
24     function getConfigurationPageUrl(context)
25     {
26       var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
27       context.configurationPageUrl = url.resolveScript({
28         scriptId: 'customscript_configuration_suitelet',
29         deploymentId: 'customdeploy_configuration_deployment',
30         params: {
31           configurationId: configurationId
32         }
33       });
34     }
35   }

```

```

36 function parseData(context)
37 {
38     var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: "configuration_id"});
39     var configuration = loadConfiguration(configurationId);
40     var data = JSON.parse(context.inputData.getContents());
41
42     /*
43     Sample data:
44
45     {
46         "accounts": [
47             {
48                 "accountId": "ACCOUNT1",
49                 "employeeId": "EMPLOYEE1",
50                 "cardHolder": "Card Holder",
51                 "dataAsOfDate": "2020-07-01",
52                 "openingBalance": 0.0,
53                 "closingBalance": 100.0,
54                 "currentBalance": 100.0,
55                 "dueBalance": 100.0,
56                 "transactions": [
57                     {
58                         "date": "2020-07-01",
59                         "amount": 100.0,
60                         "transactionTypeCode": "CHARGE",
61                         "uniqueId": "TRN001",
62                         "id": "CHK001",
63                         "payee": "A Customer",
64                         "currency": "US",
65                         "memo": "Customer Credit",
66                         "transactionStatus": "Posted",
67                         "customerReferenceId": "CUST01",
68                         "invoiceReferenceIds": ["101", "102"],
69                         "billedTaxAmount": 10.0,
70                         "localChargeAmount": 100.0,
71                         "currencyExchangeRate": 1.0,
72                         "expenseCode": "CC"
73                     }
74                 ]
75             }
76         ]
77     }
78     */
79
80     for (var accountIndex = 0; accountIndex < data.accounts.length; accountIndex++) {
81         var account = data.accounts[accountIndex];
82
83         var accountData = context.createAccountData({
84             accountId: account.accountId,
85             employeeId: account.employeeId,
86             cardHolder: account.cardHolder,
87             dataAsOfDate: account.dataAsOfDate,
88             openingBalance: account.openingBalance,
89             closingBalance: account.closingBalance,
90             currentBalance: account.currentBalance,
91             dueBalance: account.dueBalance
92         });
93
94         for (var transactionIndex = 0; transactionIndex < account.transactions.length; transactionIndex++) {
95             var transaction = account.transactions[transactionIndex];
96             accountData.createNewTransaction({
97                 date: transaction.date,
98                 amount: transaction.amount,
99                 transactionTypeCode: transaction.transactionTypeCode,
100                 uniqueId: transaction.uniqueId,
101                 id: transaction.id,
102                 payee: transaction.payee,
103                 currency: transaction.currency,
104                 memo: configuration.custrecord_memoprefix + transaction.memo,
105                 transactionStatus: transaction.transactionStatus,
106                 customerReferenceId: transaction.customerReferenceId,
107                 invoiceReferenceIds: transaction.invoiceReferenceIds,
108                 billedTaxAmount: transaction.billedTaxAmount,

```

```

109         localChargeAmount: transaction.localChargeAmount,
110         currencyExchangeRate: transaction.currencyExchangeRate,
111         expenseCode: transaction.expenseCode
112     });
113     }
114 }
115 }
116
117 function getStandardTransactionCodes(context)
118 {
119     context.createNewStandardTransactionCode({
120         transactionCode: 'CHARGE',
121         transactionType: 'PAYMENT'
122     });
123 }
124
125 function getExpenseCodes(context)
126 {
127     context.createNewExpenseCode({
128         code: 'CC',
129         description: 'Customer Credit'
130     });
131 }
132
133 return {
134     getConfigurationPageUrl: getConfigurationPageUrl,
135     parseData: parseData,
136     getStandardTransactionCodes: getStandardTransactionCodes,
137     getExpenseCodes: getExpenseCodes
138 }
139 };

```

Guidelines for Creating a Plug-in Implementation Script File

Use the following guidelines when creating the plug-in implementation script file:

- The plug-in script file can have any name, as long as it has a .js extension and contains an implementation of each of the interface functions.
- You can include utility files with helper functions when you're creating the plug-in implementation.
- Due to limitations with how strings are handled in NetSuite, you cannot use the JavaScript case statement in the plug-in implementation script file.
- The plug-in allows up to 1000 usage units.
- If the character limit is exceeded in a field, NetSuite truncates the value.

Administering a Financial Institution Parser Plug-in

To install and set up a Financial Institution Parser Plug-in, complete the following steps:

- [Enabling Features for a Financial Institution Parser Plug-in](#)
- [Adding the Plug-in Implementation](#)
- [Conditions for Running a Financial Institution Parser Plug-in](#)
- [Testing a Financial Institution Parser Plug-in](#)
- [Error Propagation](#)

Enabling Features for a Financial Institution Parser Plug-in

See [Enabling Features for a Financial Institution Parser Plug-in](#).

Adding the Plug-in Implementation

After creating the plug-in script file, upload and implement the file in NetSuite.

To add the plug-in implementation:

1. In NetSuite, go to Customization > Plug-ins > Plug-in Implementations > New.
NetSuite displays the Upload Plug-in Implementation page.
2. For the **Script File**, click the plus icon.
3. Click **Choose File**, then select the JavaScript file.
4. Click **Save**.
5. Click **Create Plug-in Implementation**.
6. On the **Plug-in Implementation** page, enter the following information:

Option	Description
Name	User-friendly name for the implementation. This name appears in the plug-in implementation list.
ID	Internal ID for the implementation for use in scripting. If you do not provide an ID, NetSuite provides one for you after you click Save . As a developer, when you write code that uses this implementation, you reference the implementation using this ID.
Status	Current status for the implementation. Choose Testing to have the implementation accessible to the owner of the implementation. Choose Released to have the implementation accessible in a production environment
Log Level	Logging level you want for the script. Select Debug, Audit, Error, or Emergency. The log information appears on the Execution Log subtab for the implementation after you create it. To see the log information later, go to Customization > Plug-ins > Plug-in Implementations, select your implementation, and then click the Execution Log subtab.
Execute As Role	Role that the script runs as. The Execute As Role field provides role-based granularity in terms of the permissions and restrictions of the executing script. The Current Role value indicates that the script executes with the permissions of the currently logged-in NetSuite user. If you created a custom role, be sure to test using that role. You can then bundle the custom role to distribute it with the plug-in implementation.
Description	Optional description of the implementation.
Owner	User account that owns the implementation. The default value is the name of the current user
Inactive	Indicates that the plug-in implementation does not run in the account. Inactivate a plug-in implementation, for example, to temporarily disable it for testing purposes.

7. On the **Unhandled Errors** subtab, select which individual or individuals NetSuite notifies if script errors occur. By default, the **Notify Script Owner** box is checked.



Tip: To enter multiple email addresses in the **Notify Emails** box, separate email addresses with a semicolon.

8. Click **Save**.

Conditions for Running a Financial Institution Parser Plug-in

To run a Financial Institution Parser Plug-in implementation, you must configure and save a format profile that uses the plug-in. You can run the plug-in using both Bank Reconciliation and Corporate Card Expenses format profiles. For details, see the help topic [Format Profile Creation](#).

Note: To perform a successful import in a Bank Reconciliation format profile, you need at least one active linked account.

When you save a format profile configured for automatic import for the first time, NetSuite creates a daily import schedule. Automatic import needs a Financial Institution Parser Plug-in and the Financial Institution Connectivity Plug-in. Together, they let you securely connect to your data source and get data. For more information, see the help topic [Automatic Bank Data Import](#).

If the profile type is Bank Reconciliation and it's your first save, NetSuite automatically downloads the last 60 days of data from your linked accounts.

After the format profile is saved for the first time, subsequent imports can occur in the following ways:

- During a scheduled import of transaction data.
- If you are importing data for bank or credit card reconciliation:
 - When you edit and save an existing format profile configured for connectivity to a financial institution (see the help topic [Format Profile Creation](#)). The plug-in runs, enabling NetSuite to parse and retrieve new data from existing accounts since the last successful import. For new accounts, NetSuite imports the last 60 calendar days of data.
 - When you manually update imported data for an account on the Match Bank Data page, the plug-in runs so NetSuite can import new data since the last successful import. The new data shows up in the Imported Bank Data grid. For details, see the help topic [Updating Imported Bank Data](#).

Note: You should only use this method in the following scenarios:

- You have a need to test the plug-in for a single financial institution account.
- The selected account does not contain the latest data.
- There was an import error.

If you're setting up a format profile for manual import of bank or credit card data, you'll need to import data on the Upload File page after setup. The plug-in runs when you click Import. For details, see the help topic [Manual Bank Data Import](#).

To verify that the plug-in is working, see [Testing a Financial Institution Parser Plug-in](#).

Testing a Financial Institution Parser Plug-in

To make sure the automatic import of transaction data works, you can test your Financial Institution Parser Plug-in implementation.

To test a Financial Institution Parser Plug-in:

1. Run a Financial Institution Parser Plug-in to import financial institution data. For details, see [Conditions for Running a Financial Institution Parser Plug-in](#).
2. View your imported transaction data. For details, see the help topic [Viewing Imported Banking Data](#).

A successful import indicates that your plug-in implementation is correctly configured.

If you receive an import error, look at the plug-in's execution log for details. Go to Customization > Plug-ins > Plug-in Implementations. Click the **View** link of the Financial Institution Parser Plug-in, and then on the Plug-in Implementation page, click the **Execution Log** subtab.

Error Propagation

You should raise all user-facing error messages using the N/error SuiteScript module. For details on this module, see the help topic [N/error Module](#).

In all user-facing errors, you must set the name property to FIPARSER_USER_ERROR. Errors not raised with the FIPARSER_USER_ERROR name property, as well as any exception not handled, are not surfaced to users. Instead, NetSuite displays a generic error message on the Format Profile page. You can find details about the error in the plug-in's execution log.

The following sample code illustrates how to raise a user-facing error:

```
1 | define(['N/error'], function (error) {
2 |     var module = {};
3 |     module.getConfigurationPageUrl = function (context) {
4 |         throw error.create({name: 'FIPARSER_USER_ERROR', message: 'User facing error'});
5 |     }
6 | });
```

The following sample code illustrates how to raise an error that is not user-facing:

```
1 | throw 'This message will not be displayed on the Format Profile page';
```

Financial Institution Parser Plug-in Interface Definition

Note: The createNewTransaction object needs to contain cardCurrency (billedCurrencyISOCode) in the additional fields to run the script correctly.

The plug-in interface includes the following interface functions, interface objects, and object functions:

Interface Function	Containing Interface Objects	Containing Object Functions
parseData()	inputData accountData pluginConfiguration	getConfigurationFieldValue() createAccountData() createNewTransaction()
getStandardTransactionCodes()		createNewStandardTransactionCode()
getExpenseCodes()		createNewExpenseCode()
getConfigurationPageUrl()	pluginConfiguration	getConfigurationFieldValue() configurationPageUrl

Note: The intention of the Financial Institution Parser Plug-in interface is not to separate implementations for bank reconciliation and corporate card expense configuration. The usage of imported data is determined by the configuration of the format profile record.

parseData()

Function Declaration	function parseData(context)
Type	Interface function
Description	Transform a data file into account and transaction data.
Returns	void
Input Parameters	context – a JavaScript object with the following properties: - inputData - pluginConfiguration - createAccountData()

Example

```

1 function parseData(context) {
2   var input = context.inputData;
3   input.lines.iterator().each(function (line) {

```

```

4 |     var accountData = context.createAccountData({ ... });
5 |     accountData.createNewTransaction({ ... });
6 | }
7 | }

```

inputData

Type	Interface object
Description	Access the input data to be parsed. inputData is an nlobjFileInterface, which is a file object that contains the bank statement content. You can also use a file API to stream large files. For more information on streaming large files, see the help topic N/file Module.

Example

```

1 | var input = context.inputData;
2 | input.lines.iterator().each(function (line) {
3 |     ...
4 | }

```

pluginConfiguration

Type	Interface object
Description	Allow the plug-in to retrieve user-supplied standard configuration properties (field values) for this plug-in.
Object Function(s)	getConfigurationFieldValue(options)
Parent Object(s)	parseData()
Child Object(s)	N/A

getConfigurationFieldValue()

Function Declaration	function getConfigurationFieldValue(options)
Type	Object function
Description	Retrieve a named standard configuration value. For all Financial Institution Parser Plug-ins that you develop, the configuration_id value is available. This is a unique identifier at the format profile level. It can be used to refer to a format profile, even if the record is not created.
Returns	String
Input Parameters	options – a JavaScript object with the property FieldName, which is a string. This property is required and refers to the name of the configuration property to retrieve.

Example

```
1 | var configurationId = context.pluginConfiguration.getConfigurationFieldValue({fieldName: 'configuration_id'});
```

createAccountData()

Function Declaration `function createAccountData(options)`

Type Object function

Description Create a new account data set.

Returns [accountData](#)

options is a JavaScript object with the following input parameters:

Parameter	Type	Required	Description	Since
accountId	String	Yes, for bank reconciliation	The raw ID of the external account, which the plug-in uses to link the imported account to a valid account in NetSuite.	2020.2
employeeId	String	May be required for employee expenses	The NetSuite employee ID, if provided. To link corporate card expense accounts to employee accounts, the employee ID or cardholder name must be provided.	2020.2
cardHolder	String	May be required for employee expenses	The cardholder name for credit cards. To link corporate card expense accounts to employee accounts, the cardholder name or employee ID must be provided.	2020.2
dataAsOfDate	String	No	The date when data from a financial services provider was last updated. This can correspond to a file date, statement date, or other declared "last updated" timestamp from the data source. NetSuite currently accepts the ISO 8601 extended local date format. When dataAsOfDate is set, the following fields are populated from the most recent imported account statement containing a statement date: ■ Bank Balance and Balance As Of on the Match Bank Data page ■ Statement End Date and Ending Statement Balance on the Reconcile Account Statement page If no account statements contain a statement date, the following takes place: ■ The Bank Balance and Balance As Of fields are not populated. ■ The Statement End Date and Ending Statement Balance fields are populated with	2020.2

Parameter	Type	Required	Description	Since
			the current date, and the closing balance from the last reconciliation statement, respectively.  Note: If you use a default parser to upload a file without an account statement date, NetSuite uses the most recent imported transaction date to set the statement date. This date may or may not correspond to the real account statement date. The date must be in the format YYYY-MM-DD. For example, 2018-05-28.	
openingBalance	Number	No	The amount of money in the account at the start of an accounting period.	2020.2
closingBalance	Number	No	The amount of money left in the account at the end of an accounting period.	2020.2
currentBalance	Number	No	The current amount of money in the account.	2020.2
dueBalance	Number	No	The amount of money owed on an account.	2020.2

Example

```

1  var account = context.createAccountData({
2      accountId: '101',
3      account: '5430001',
4      account: 'ALICE GREEN',
5      dataAsOfDate: '2019-12-19',
6      openingBalance: 100.0,
7      closingBalance: 120.0,
8      currentBalance: 125.0,
9      dueBalance: 10.0
10 });
11
12 var transaction = account.createNewTransaction({...});

```

accountData

Type Interface object

Description The context for an account for which transactions may be created.

Object Function(s) [createNewTransaction\(\)](#)

Parent Object(s) N/A

Child Object(s) N/A

createNewTransaction()

Function Declaration `function createNewTransaction(options)`

Type Object function

Description Add a new imported transaction to the account data set.

Returns void

options is a JavaScript object with the following input parameters:

Parameter	Type	Required	Description	Since
date	String	Yes	The date of the transaction. NetSuite currently accepts the ISO 8601 extended local date format.	2020.2
amount	Number	Yes	The amount of the transaction.	2020.2
transactionTypeCode	String	Yes, for bank reconciliation	The raw bank transaction code NetSuite uses to map to a valid bank data type in NetSuite. You can set a transaction code from the Code Type Mapping subtab on the Format Profile page, or through SuiteScript using the <code>getStandardTransactionCodes(context)</code> interface function.	2020.2
uniqueId	String	No	The unique identifier of a transaction, which is used to detect duplicate transactions. The uniqueId automatically prevents a transaction from getting imported into NetSuite as a duplicate.	2020.2
id	String	No	The transaction number. For example, a cash sale transaction could have a transaction number of CS0001.	2020.2
payee	String	No	The payee to whom the transaction is being sent.	2020.2
currency	String	No	One of the standard currency codes to which the user has access.	2020.2
memo	String	No	The memo of the transaction.	2020.2
transactionStatus	String	No	The status of the transaction.	2020.2
customerReferenceId	String	No	The customer ID for the transaction.	2020.2
customerName	String	No	The customer name for the transaction.	2020.2
invoiceReferenceIds	Array[String]	No	The list of invoice reference numbers for the transaction.	2020.2
billedTaxAmount	Number	Yes, for employee expenses	The amount of the billed tax in the account's billing currency.	2020.2
localChargeAmount	Number	Yes, for employee expenses	The transaction amount in local currency.	2020.2
localTaxAmount	Number	Yes, for employee expenses	The amount of the billed tax in local currency.	2020.2
currencyExchangeRate	Number	Yes, for employee expenses	The exchange rate between local currency and the account's billing currency.	2020.2

Parameter	Type	Required	Description	Since
			The exchange rate on the transaction is based on the capture date. The billing amount is divided by the local amount.	
expenseCode	String	Yes, for employee expenses	The raw expense code NetSuite uses to map to an expense category in NetSuite. You can set an expense code from the Expense Code Mapping subtab on the Format Profile page, or through SuiteScript using the <code>getExpenseCodes(context)</code> interface function.	2020.2

Example

```

1 var transaction = account.createNewTransaction({
2   date: '2019-12-19',
3   amount: 120.0,
4   transactionTypeCode: 'P',
5   uniqueId: '1043423',
6   id: '243',
7   payee: 'Oracle Cloud',
8   currency: 'USD',
9   memo: 'invoice 123',
10  transactionStatus: 'Cancelled',
11  customerReferenceId: '34',
12  customerName: 'Amit',
13  invoiceReferenceIds: ['100', '101'],
14  billedTaxAmount: 15.0,
15  localChargeAmount: 100.0,
16  localTaxAmount: 17.0,
17  currencyExchangeRate: 1.13,
18  expenseCode: '4'
19 });

```

getStandardTransactionCodes()

Function Declaration	function getStandardTransactionCodes(context)
Type	Interface function
Description	Retrieve transaction codes that are standard for this data format. On the format profile record, the Code Type Mapping subtab displays a grid where you can provide custom transaction codes and map them to corresponding bank data types in NetSuite. For details, see the help topic Mapping Custom Bank Codes.
Returns	void
Input Parameters	context - a JavaScript object with the property <code>createNewStandardTransactionCode()</code> .

Example

```

1 function getStandardTransactionCodes(context) {
2   context.createNewStandardTransactionCode({
3     transactionCode: 'CREDIT',

```

```

4 |         transactionType: 'DEPOSIT'
5 |     });
6 | }

```

createNewStandardTransactionCode()

Function Declaration	function createNewStandardTransactionCode(options)
Type	Object function
Description	Create a new StandardTransactionCode object for the plug-in to set property values.
Returns	void

options is a JavaScript object with the following input parameters:

Parameter	Type	Required	Description	Since
transactionCode	String	Yes	The raw transaction code from the parsed transaction.	2020.2
transactionType	String	Yes	The bank data type in NetSuite to which the code maps, which can be one of the following: - ACH - CHECK - CREDIT - CREDIT_OR_DEBIT - DEBIT - DEPOSIT - FEE - INTEREST - PAYMENT - TRANSFER - OTHER	2020.2
creditDebit	String	No	Used by BAI2 or a similar file format to determine whether the amount should be positive or negative. This parameter can be one of the following: - CREDIT - DEBIT	2020.2
description	String	No	A description of the kind of transaction that the transaction code represents.	2020.2

Example

```

1 | var code = context.createNewStandardTransactionCode({
2 |     transactionCode: 'P',
3 |     transactionType: 'PAYMENT',
4 |     creditDebit: 'DEBIT',
5 |     description: 'Payment'
6 | });

```

getExpenseCodes()

Function Declaration	function getExpenseCodes(context)
Type	Interface function
Description	Retrieve expense codes as defined by the file format being used. When a format profile record is configured to import corporate card expenses, the Expense Code Mapping subtab appears, which contains a grid populated with values returned by getExpenseCodes(). This subtab enables you to map expense codes to expense categories to automatically populate expense categories in expense reports.

 **Note:** You cannot add custom expense codes.

For more information, see the help topic [Mapping Custom Expense Codes](#).

Returns	void
Input Parameters	context - a JavaScript object with the property createNewExpenseCode() .

Example

```

1 function getExpenseCodes(context) {
2     context.createNewExpenseCode({
3         code: '4',
4         description: 'RESTAURANT'
5     });
6 }
```

createNewExpenseCode()

Function Declaration	function createNewExpenseCode(options)
Type	Object function
Description	Create a new ExpenseCode object for the plug-in to set property values.
Returns	void

options is a JavaScript object with the following input parameters:

Parameter	Type	Required	Description	Since
code	String	Yes	A raw expense code supported by the data file.	2020.2
description	String	No	A description of the kind of expense that the expense code represents.	2020.2

Example

```

1 var expenseCode = context.createNewExpenseCode({
2     code: '4',
```

```

3 |     description: 'RESTAURANT'
4 | });

```

getConfigurationPageUrl()

Function Declaration `function getConfigurationPageUrl(context)`

Type Interface function

Description Access the user interface for configuring the plug-in. For most plug-in authors, this is a URL to a Suitelet. This URL appears on the Parser Configuration subtab on a format profile record.

Returns void

Input Parameters context - a JavaScript object with the following properties:

- `pluginConfiguration`
- `configurationPageUrl`

Example

Note: You should define a custom record to store your configuration for setting up a bank connection. You should then deploy a Suitelet to create a custom implementation of a UI to save and load the record. The Suitelet lets you implement a custom interface that enables you to give NetSuite access to your bank accounts.

When returning a URL to the deployed Suitelet (using the `getConfigurationPageUrl` method), do not hardcode the URL. Instead, use the standard `resolveScript` SuiteScript method to define the structure of the URL, which includes specifying the script ID, deployment ID, and parameters.

For information about Suitelets, see the help topic [SuiteScript 2.x Suitelet Script Type](#). To build a UI in a Suitelet, see the help topic [N/ui/serverWidget Module](#).

```

1 | function getConfigurationPageUrl(context) {
2 |     var configurationId = context.pluginConfiguration.getConfigurationFieldValue({
3 |         name: 'configuration_id'
4 |     });
5 |     context.configurationPageUrl = url.resolveScript({
6 |         scriptId: 'customscript_configuration_suitelet',
7 |         deploymentId: 'customdeploy_configuration_deployment',
8 |         params: {
9 |             configurationId: configurationId
10 |         }
11 |     });
12 | }

```

pluginConfiguration

See [pluginConfiguration](#).

getConfigurationFieldValue()

See [getConfigurationFieldValue\(\)](#).

configurationPageUrl

Type	Interface object
Description	Set the URL to use for parser configuration.
Object Function(s)	N/A
Parent Object(s)	getConfigurationPageUrl()
Child Object(s)	N/A

For an example in which the property configurationPageUrl is used, see [getConfigurationPageUrl\(\)](#).